



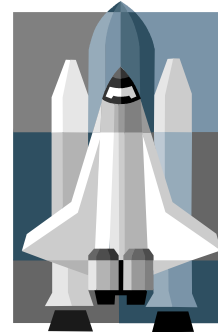
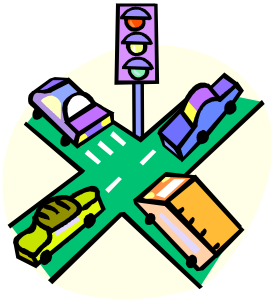
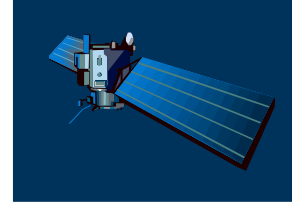
JOHNS HOPKINS
UNIVERSITY

EN.601.422 / EN.601.622

Software Testing & Debugging

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

Software is everywhere



Do you see any bugs?

```
public class NumFinder {  
    private int smallest = Integer.MAX_VALUE;  
    private int largest = Integer.MIN_VALUE;  
  
    public void find(int[] nums) {  
        for(int n : nums) {  
            if (n < smallest)  
                smallest = n;  
            else if (n > largest)  
                largest = n;  
        }  
    }  
    // getters for smallest and largest  
}
```

Terminology

- ▶ **Fault:** a static defect in the software
- ▶ **Error:** An incorrect internal state
- ▶ **Failure:** External incorrect behavior with respect to the requirements or expected behavior
- ▶ **Bug:** an informal term used in place of any of the above
- ▶ **Debugging:** Given a failure, find/fix the fault

Fault and Failure Analogy

- ▶ A patient gives a doctor a list of symptoms
 - ❖ **Failures**
- ▶ The doctor tries to diagnose the root cause, the ailment
 - ❖ **Fault**
- ▶ The doctor may look for anomalous internal conditions (high blood pressure, irregular heartbeat, bacteria in the blood stream)
 - ❖ **Errors**

Fault vs Error vs Failure

```
public static int numZero (int [ ] arr)
{ // Effects: If arr is null throw NullPointerException
  // else return the number of occurrences of 0 in arr
  int count = 0;
  for (int i = 1; i < arr.length; i++)
  {
    if (arr[i] == 0)
      count++;
  }
  return count;
}
```

Fault: i should start from 0 not 1

Error: i is 1, not 0, on
the first iteration

Failure: None

Error: i is 1, not 0, on
The error propagates to count
Failure: count is 0 at the end

Test 1

[2, 7, 0]

Expected: 1

Actual: 1

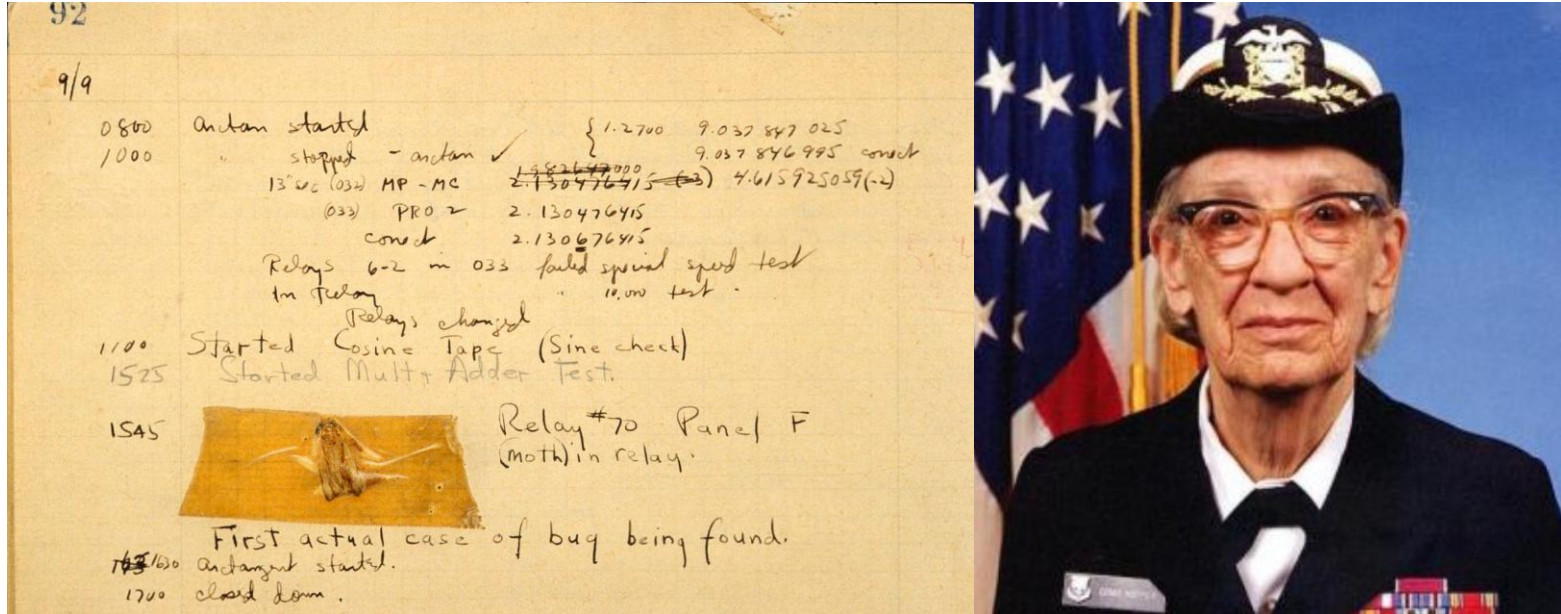
Test 2

[0, 2, 7]

Expected: 1

Actual: 0

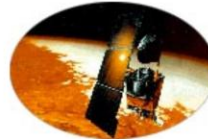
First Ever Bug



Grace Hopper found a moth stuck between the relays on the Harvard Mark II computer she was working on

Mars Climate Orbiter Loss

► https://en.wikipedia.org/wiki/Mars_Climate_Orbiter



November 10, 1999 MCO Failure Board Releases Report

Wide-ranging managerial and technical actions are

underway at NASA's Jet Propulsion Laboratory, Pasadena, CA, in response to the loss of the Mars Climate Orbiter and the initial findings of the mission failure investigation board. [Full Story](#)

SEPTEMBER 30, 1999 Likely Cause Of Orbiter Loss Found

The peer review preliminary findings indicate that one team used English units (e.g., inches, feet and pounds) while the other used metric units for a key spacecraft operation. [Full Story](#)

SEPTEMBER 24, 1999

Therac-25 Radiation Therapy Machine

- ▶ Therac-25: the killer machine!
 - ❖ race conditions in the codebase led to excessive radiations and death of three patients!



Ariane 5 Rocket Explosion on June 4, 1996



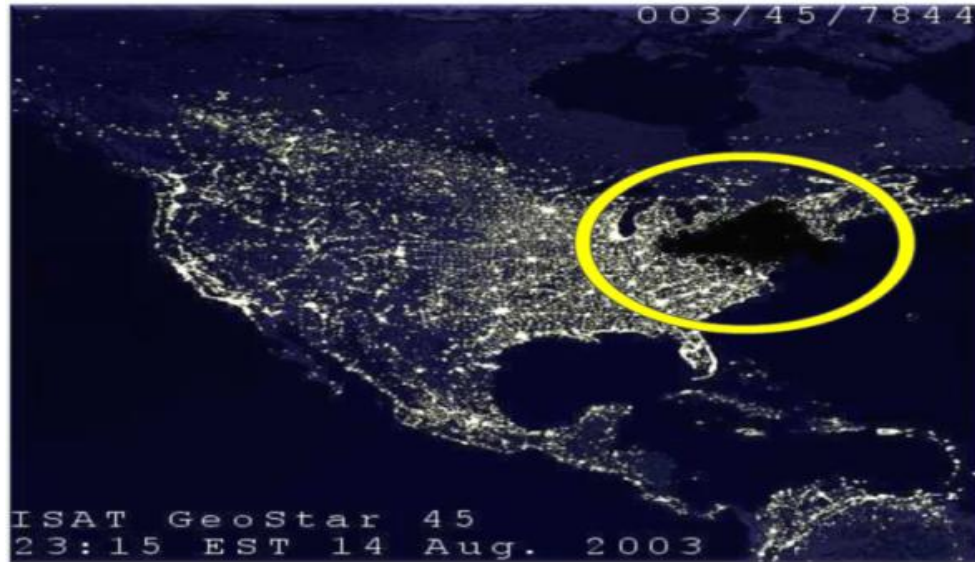
https://www.youtube.com/watch?v=PK_yguLapgA

Ariane 5 Rocket Explosion on June 4, 1996

- ▶ **Cause:** a 64-bit floating-point number relating to the horizontal velocity of the rocket with respect to the platform was converted to a 16-bit signed integer causing a data overflow.
- ▶ **Estimated Cost:** a decade of development costing \$7 billion

Northeast Blackout of 2003

- ▶ https://en.wikipedia.org/wiki/Northeast_blackout_of_2003
- ▶ The outage affected about 10 million people in Canada and about 40 million in eight U.S. states.



Northeast Blackout of 2003

- ▶ **Cause:** software bug in the alarm system in the control room of FirstEnergy, an Akron Ohio–based company, which rendered operators unaware of the need to redistribute load after overloaded transmission lines drooped into foliage. What should have been a manageable local blackout cascaded into collapse of the entire Northeast region.
- ▶ **Estimated Cost:** contributing to 11 deaths and about \$6 billion USD

Toyota Prius ABS Brake Software Recall

► <https://www.usnews.com/news/articles/2014/02/12/toyota-to-recall-19-million-prius-cars-due-to-software-problem>

Toyota to Recall 1.9 Million Prius Cars Due to Software Problem

The company says a software problem can cause the vehicles to stop while they are being driven.

By Danielle Kurtzleben, Staff Writer Feb. 12, 2014, at 11:06 a.m.

f t y e ...



Toyota is recalling nearly 2 million of its Prius hybrids. DAVID ZALUBOWSKI/AP

Korean Education Software Fault

- ▶ <http://koreajoongangdaily.joins.com/news/article/article.aspx?aid=2939367>
- ▶ In 2011, Samsung Electronics' Education info system miscalculated grades of about 29000 middle and high students, affecting the college admission process leading to a government investigation

Robinhood Bugs

Forbes

20-Year-Old Robinhood Customer Dies By Suicide After Seeing A \$730,000 Negative Balance

With additional reporting by John Dobosz and Jeff Kauflin

The note found on his computer by his parents on June 12, 2020, asked a simple question. “How was a 20 year old with no income able to get assigned almost a million dollars worth of leverage?” The tragic message was written by Alexander E. Kearns, a 20-year-old student at the University of Nebraska, home from college and living with his parents in Naperville, Illinois. Earlier that



Alexander E. Kearns. KEARNS FAMILY

INSIDER

MARKETS
INSIDER

Copycat traders are one-upping one another as they exploit Robinhood's 'infinite money' glitch — and one user now holds a \$1.7 million portfolio

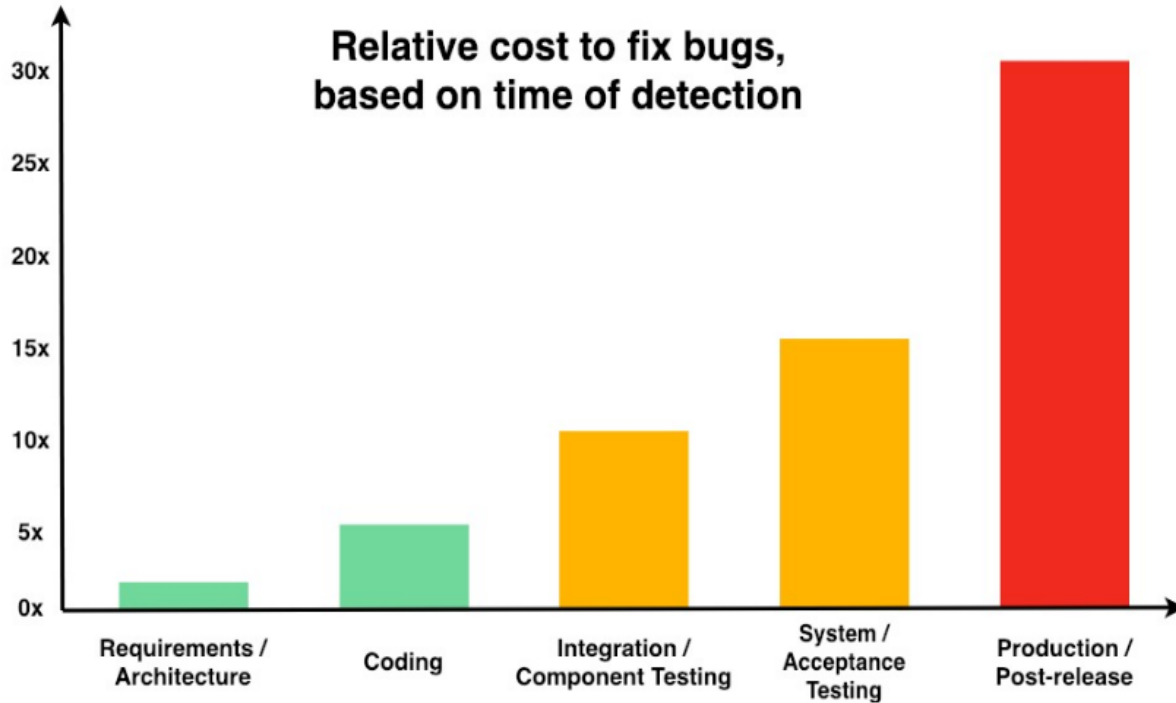
Ben Winck Nov 6, 2019, 12:59 PM



Importance of Software Testing

- ▶ A 2002 report by NIST:
 - ❖ Defective software costs US economy \$59.5 billion per year
 - ❖ Improved testing could reduce the cost by one third
- ▶ Blumenstyk reported web application failures cost:
 - ❖ \$150K in media companies
 - ❖ \$2.4 million per hour in credit card sales
 - ❖ \$6.5 million per hour in financial service market

Cost of Bugs!



“Most defects end up costing more than it would have cost to prevent them. Defects are expensive when they occur, both the direct costs of fixing the defects and the indirect costs because of damaged relationships, lost business, and lost development time.” —*Kent Beck, Extreme Programming Explained*

What Do All These Mean?

Software Testing is a crucial activity

The Goal of Software Testing

Why do we test software?

Ensuring that the software works as expected

The Goal of Software Testing

Why do we test software?

~~Ensuring that the software works as expected~~

The Goal of Software Testing

Why do we test software?

To produce high quality software by revealing
and repairing as many faults as possible



The Goal of Software Testing

Why do we test software?

“Software testing should be used to show the presence of bugs, not their absence”

– Edward W. Dijkstra

The Goal of Software Testing

► Issue with the “common sense” goal:

show correctness → impossible/impractical to achieve, due to (potentially) infinite number of possible test input values

Psychology of Software Testing

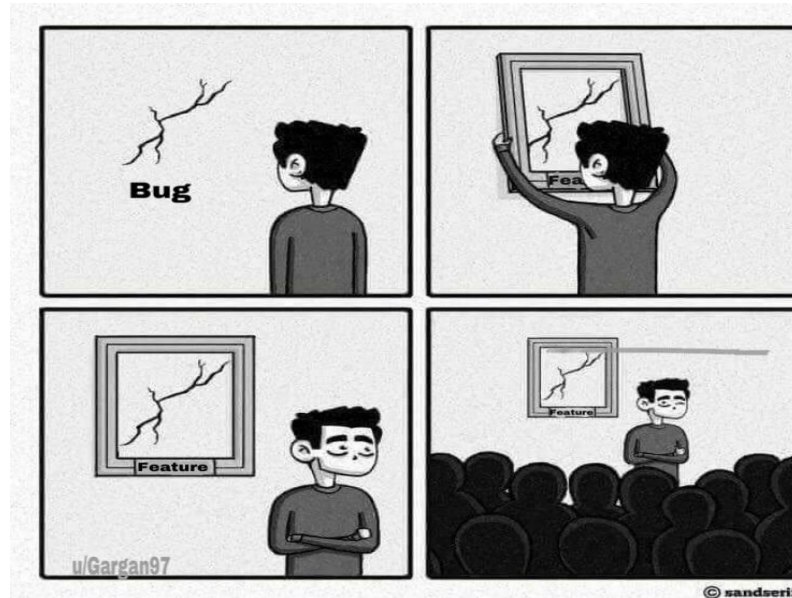
- ▶ Psychological issue of the “common sense” goal:

show correctness → will be inclined (subconsciously) towards your goal → write tests that work fine rather than cause failures

Psychology of Software Testing

- ▶ A fundamental issue of the “common sense” goal:

show correctness ➔ what does “correctness” really mean?



The Goal of Software Testing

show correctness → impossible to achieve, due to (potentially) infinite number of possible test input values

exhaustive path testing, via adequate test input values, might seem to be the answer
(i.e., executing all possible paths of control flow graph)

The Goal of Software Testing

- ▶ Why exhaustive path testing is not the answer either:
 - ❖ Even if we cover all the possible execution paths, the program might be loaded with faults:
 - no guarantee the program complies with its specifications
 - missing paths
 - data sensitivity:

```
if (a - b < c)
    System.out.println ("a distance from b is less than c");
```

 - * Test 1: [a=0, b=2, c=1] executes inside the “if” but does not reveal the fault (i.e., must calculate absolute distance between a and b)

Psychology of Software Testing

- ▶ Implications of the “correct” perspective:
 - ❖ testing is difficult
 - ❖ switch successful test vs unsuccessful test terminology
 - to our thinking, a successful test should reveal a fault
 - ❖ A program that does everything it is supposed to do is not necessarily correct
 - might do extra unnecessary things also

Testing is a destructive (even sadistic!)
process

A major shift in our mentality!

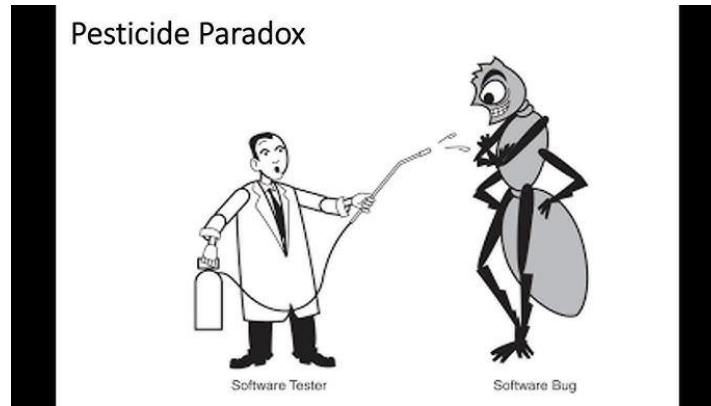
Psychology of Software Testing

eventually we want to increase the confidence about the software

This is best achieved by diligent
search for faults

Software Testing is hard!

Pesticide Effect: When the same set of test cases is used repeatedly, over time they become less effective at finding new defects, because the software eventually “learns” to pass those tests. The term is borrowed from agriculture: like pests resisting the same pesticide, software tests need to be updated!



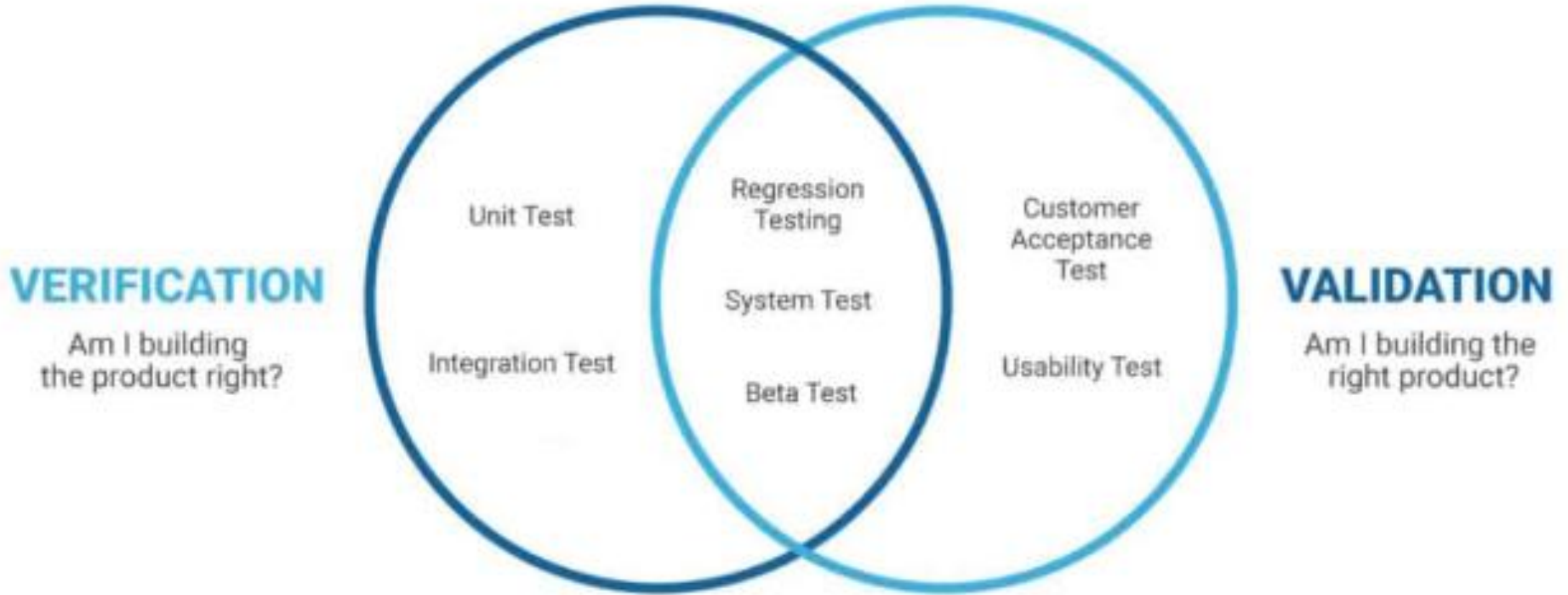
Software Testing is hard!

- ▶ Absence-of-errors fallacy: Testing software thoroughly but for wrong requirements!
- ▶ Testing is context-dependent:
 - ❖ Mobile vs. web vs. desktop

Verification & Validation

- ▶ **Verification:** determining whether the software fulfills the requirements. In other words, whether the software meets its specifications or not.
 - ❖ Are we building the product right?
- ▶ **Validation:** determining whether the produced meets the intended usage (i.e., comprehensively and exclusively captures user needs)
 - ❖ Are we building the right product?

Verification vs Validation



Verification & Validation

► **Verification:**

- ❖ more technical
- ❖ does not reveal flaws in the requirements/specifications

► **Validation:**

- ❖ depends on the domain knowledge
- ❖ exposes flaws in the specification and requirement deficiencies/discrepancies